

Name: Kriti

College: Lovely Professional University

Week6 Assignment

Email: kritijulia@gmail.com

StudentId:CT_CSI_DV_5113

Q1. Deploy Replica Set and Replication Controller, and deployment. Also learn the advantages and disadvantages of each.

Answer:

Feature	ReplicationController	ReplicaSet	Deployment
Introduced in	Kubernetes v1 (legacy)	Replacement for RC	Manages ReplicaSets
Rolling Updates	NO	NO	Yes
Rollbacks	NO	NO	Yes
Label Selector	Equality only	Set-based	Set-based
Usage in Modern K8s	Deprecated	Rarely used directly	Standard
Versioning Support	NO	NO	Yes

1. ReplicaSet (RS) : Ensures that a specified number of **pod replicas** are running at any given time.
 - a. Advantages:
 - i. Maintains the **desired number of pod replicas**.
 - ii. Supports **set-based label selectors** (more powerful than ReplicationController).
 - b. Disadvantages:
 - i. **Cannot handle rolling updates or rollbacks**.
 - ii. Not typically used directly — usually managed through a Deployment.
 - c. Commands:
 - i. Deploy: `kubectl apply -f nginx-replicaset.yaml`
 - ii. Check status: `kubectl get rs`
 - iii. Delete: `kubectl delete -f nginx-replicaset.yaml`
2. ReplicationController (RC): The **older** version of ReplicaSet, used to ensure a specific number of pod replicas are running.
 - a. Advantages:
 - i. Ensures **availability** by maintaining pod replicas.
 - ii. Simpler and well-supported in older Kubernetes versions

b. Disadvantages:

- i. Limited **label selector** support (only equality-based)
- ii. **.Deprecated** in favor of ReplicaSet and Deployment.
- iii. **No support for rolling updates or rollbacks.**

c. Commands:

- i. Deploy: `kubectl apply -f nginx-rc.yaml`
- ii. Check status: `kubectl get rc`
- iii. Delete: `kubectl delete -f nginx-rc.yaml`

3. Deployment: Manages ReplicaSets and provides **rolling updates**, **rollback**, and **version control** for applications.

a. Advantages:

- i. **Rolling updates** without downtime.
- ii. **Rollbacks** to previous versions easily.
- iii. Manages **ReplicaSets** automatically.
- iv. Preferred method for most use cases.

b. Disadvantages:

- i. Slightly more complex than directly using ReplicaSet.
- ii. Not suitable if fine-grained control of ReplicaSet is required.

c. Commands:

- i. Deploy: `kubectl apply -f nginx-deployment.yaml`
- ii. Check status: `kubectl get deploy`
- iii. Delete: `kubectl delete -f nginx-deployment.yaml`

The screenshot shows a VS Code editor with a file explorer on the left displaying a project structure for 'celestial-intern'. The main editor window shows a YAML file named 'example-voting-app.yaml' with the following content:

```
apiVersion: v1
kind: Service
metadata:
  name: example-voting-app
spec:
  selector:
    app: example-voting-app
  ports:
    - port: 80
      targetPort: 8080
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: example-voting-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: example-voting-app
  template:
    metadata:
      labels:
        app: example-voting-app
    spec:
      containers:
        - name: vote
          image: postgres:15-alpine
          environment:
            POSTGRES_USER: "postgres"
            POSTGRES_PASSWORD: "postgres"
          volumeMounts:
            - name: db-data
              mountPath: /var/lib/postgresql/data
          healthcheck:
            test: ["CMD-SHELL", "pg_isready -U postgres -d postgres"]
            interval: 10s
            timeout: 5s
            successThreshold: 1
            failureThreshold: 3
        - name: db
          image: postgres:15-alpine
          environment:
            POSTGRES_USER: "postgres"
            POSTGRES_PASSWORD: "postgres"
          volumeMounts:
            - name: db-data
              mountPath: /var/lib/postgresql/data
          healthcheck:
            test: ["CMD-SHELL", "pg_isready -U postgres -d postgres"]
            interval: 10s
            timeout: 5s
            successThreshold: 1
            failureThreshold: 3
        - name: result
          image: postgres:15-alpine
          environment:
            POSTGRES_USER: "postgres"
            POSTGRES_PASSWORD: "postgres"
          volumeMounts:
            - name: db-data
              mountPath: /var/lib/postgresql/data
          healthcheck:
            test: ["CMD-SHELL", "pg_isready -U postgres -d postgres"]
            interval: 10s
            timeout: 5s
            successThreshold: 1
            failureThreshold: 3
        - name: worker
          image: postgres:15-alpine
          environment:
            POSTGRES_USER: "postgres"
            POSTGRES_PASSWORD: "postgres"
          volumeMounts:
            - name: db-data
              mountPath: /var/lib/postgresql/data
          healthcheck:
            test: ["CMD-SHELL", "pg_isready -U postgres -d postgres"]
            interval: 10s
            timeout: 5s
            successThreshold: 1
            failureThreshold: 3
```

The terminal window at the bottom shows the output of the command `kubectl create -f demoapplication/`. The output indicates that the deployment was successful and that the service was created. The terminal also shows the command `kubectl get deploy` and its output, which lists the deployment 'example-voting-app' with 3 replicas.

Navigator: Local Kubeconfigs, aksdemocluster, Overview, Applications, Nodes, Workloads, Overview, Pods, Deployments, Daemon Sets, Stateful Sets

Welcome | Pods - aksdemocluster | 5 items

	Name	Namespace	Con...	CPU	Memor	Restart	Controlled E	Node	QoS	Age	Status
	db-745	default		N/A	N/A	0	ReplicaSet	aks-agent-0	BestEffort	46s	Running
	redis-6	default		N/A	N/A	0	ReplicaSet	aks-agent-0	BestEffort	45s	Running
	result-6	default		N/A	N/A	0	ReplicaSet	aks-agent-0	BestEffort	45s	Running
	vote-5c	default		N/A	N/A	0	ReplicaSet	aks-agent-0	BestEffort	45s	Running
	worker	default		N/A	N/A	0	ReplicaSet	aks-agent-0	BestEffort	44s	Running

```

1 apiVersion: apps/v1
2 kind: ReplicaSet
3 metadata:
4   name: frontend
5   labels:
6     app: guestbook
7     tier: frontend
8 spec:
9   # Number of replicas to maintain
10  replicas: 3
11  selector:
12    matchLabels:
13      tier: frontend
14  template:
15    metadata:
16      labels:
17        tier: frontend
18    spec:
19      containers:
20        - name: php-redis
21          image: gcr.io/google-samples/gb-frontend:v3

```

```

1 apiVersion: v1
2 kind: ReplicationController
3 metadata:
4   name: nginx
5 spec:
6   replicas: 3
7   selector:
8     app: nginx
9   template:
10     metadata:
11       name: nginx
12       labels:
13         app: nginx
14     spec:
15       containers:
16         - name: nginx
17           image: nginx
18           ports:
19             - containerPort: 80
20

```

```

1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: nginx-deployment
5   labels:
6     app: nginx
7 spec:
8   replicas: 3
9   selector:
10     matchLabels:
11       app: nginx
12   template:
13     metadata:
14       labels:
15         app: nginx
16     spec:
17       containers:
18         - name: nginx
19           image: nginx:1.14.2
20           ports:
21             - containerPort: 80

```

Navigator: Local Kubeconfigs, aksdemocluster, Overview, Applications, Nodes, Workloads, Overview, Pods, Deployments, Daemon Sets, Stateful Sets, Replica Sets, Replication Controllers, Jobs, Cron Jobs, Config, T2 Network, Storage, Namespaces, Events, Helm, Charts, Releases, Access Control, Custom Resources, Definitions, azure.com, eraser.sh, snapshot.storage.k8s.io, demo-k8s

Welcome | Pods - aksdemocluster | 8 items

	Name	Namespace	Con...	CPU	Memor	Restart	Controlled E	Node	QoS	Age	Status
	db-74574d6dd-5g	default		0.000	36.8MiB	0	ReplicaSet	aks-agent-0	BestEffort	18m	Running
	nginx-deployment-	default		0.000	0	0	ReplicaSet	aks-agent-0	BestEffort	34s	Running
	nginx-deployment-	default		0.000	0	0	ReplicaSet	aks-agent-0	BestEffort	34s	Running
	nginx-deployment-	default		0.000	0	0	ReplicaSet	aks-agent-0	BestEffort	34s	Running
	redis-6c3fb9c4b7-2	default		0.010	9.1MiB	0	ReplicaSet	aks-agent-0	BestEffort	18m	Running
	result-5f985487c-k	default		0.000	22.5MiB	0	ReplicaSet	aks-agent-0	BestEffort	18m	Running
	vote-5d74dc07c-n	default		0.000	98.4MiB	0	ReplicaSet	aks-agent-0	BestEffort	18m	Running
	worker-6f5fcd3d56	default		0.010	43.9MiB	0	ReplicaSet	aks-agent-0	BestEffort	17m	Running

Terminal: Create resource: x +

```

1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: nginx-deployment
5   labels:
6     app: nginx
7 spec:
8   replicas: 3
9   selector:
10     matchLabels:
11       app: nginx
12   template:
13     metadata:
14       labels:
15         app: nginx
16     spec:
17       containers:
18         - name: nginx

```

You're using Linux Personal (the individual or companies with < \$10M annual revenue or funding) | aksdemocluster v1.32.0

Deployments, Daemon Sets, Stateful Sets, Replica Sets, Replication Controllers, Jobs, Cron Jobs, Config, T2 Network, Storage

Scale Deployment default/nginx-deployment

Current replica scale: 3
Desired number of replicas: 6

Cancel Scale

3m 45s ago 2023-07-11T22:36:17+09:00

Navigation: Local Kubernetes, Overview, Applications, Nodes, Workloads, Overview, Pods, Deployments, Daemon Sets, Stateful Sets, Replica Sets, Replication Controllers, Jobs, Cron Jobs, Config, Network, Storage, Namespaces, Events, Helm, Charts, Releases, Access Control, Custom Resources, Definitions, acn.azure.com, eraser.sh, snapshot.storage.k8s.io, demo-k8s.

Deployments - aksdemocluster... 6 items

Name	Namespace	Pods	Replicas	Age	Status
nginx-deployment	default	3/3	3	53s	Running
redis	default	1/1	1	11m	Running
result	default	1/1	1	11m	Running
vote	default	1/1	1	11m	Running
worker	default	1/1	1	11m	Running

Terminal: Create resource x +

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: nginx-deployment
5   labels:
6     app: nginx
7 spec:
8   replicas: 3
9   selector:
10    matchLabels:
11      app: nginx
12   template:
13     metadata:
14       labels:
15         app: nginx
16     spec:
17       containers:
18         - name: nginx
```

You're using Lens Personal (for individuals or companies with < \$10M annual revenue or funding) · aksdemocluster v1.32.16

Navigation: Local Kubernetes, Overview, Applications, Nodes, Workloads, Overview, Pods, Deployments, Daemon Sets, Stateful Sets, Replica Sets, Replication Controllers, Jobs, Cron Jobs, Config, Network, Storage, Namespaces, Events, Helm, Charts, Releases, Access Control, Custom Resources, Definitions, acn.azure.com, eraser.sh, snapshot.storage.k8s.io, demo-k8s.

Stateful Sets - aksdemocluster... 1 item

Name	Namespace	Pods	Replicas	Age
web	default	0/1	1	18s

Terminal: Create resource x +

```
10 spec:
11   template:
12     metadata:
13       labels:
14         app: web
15     spec:
16       terminationGracePeriodSeconds: 10
17       containers:
18         - name: nginx
19           image: registry.k8s.io/nginx-alpine
20           ports:
21             - containerPort: 80
22           volumeMounts:
23             - name: web
24               mountPath: /usr/share/nginx/html
25       volumeClaimTemplates:
26         - metadata:
27             name: web
28           spec:
```

Q2. Kubernetes service types (ClusterIP, NodePort, LoadBalancer)

Answer:

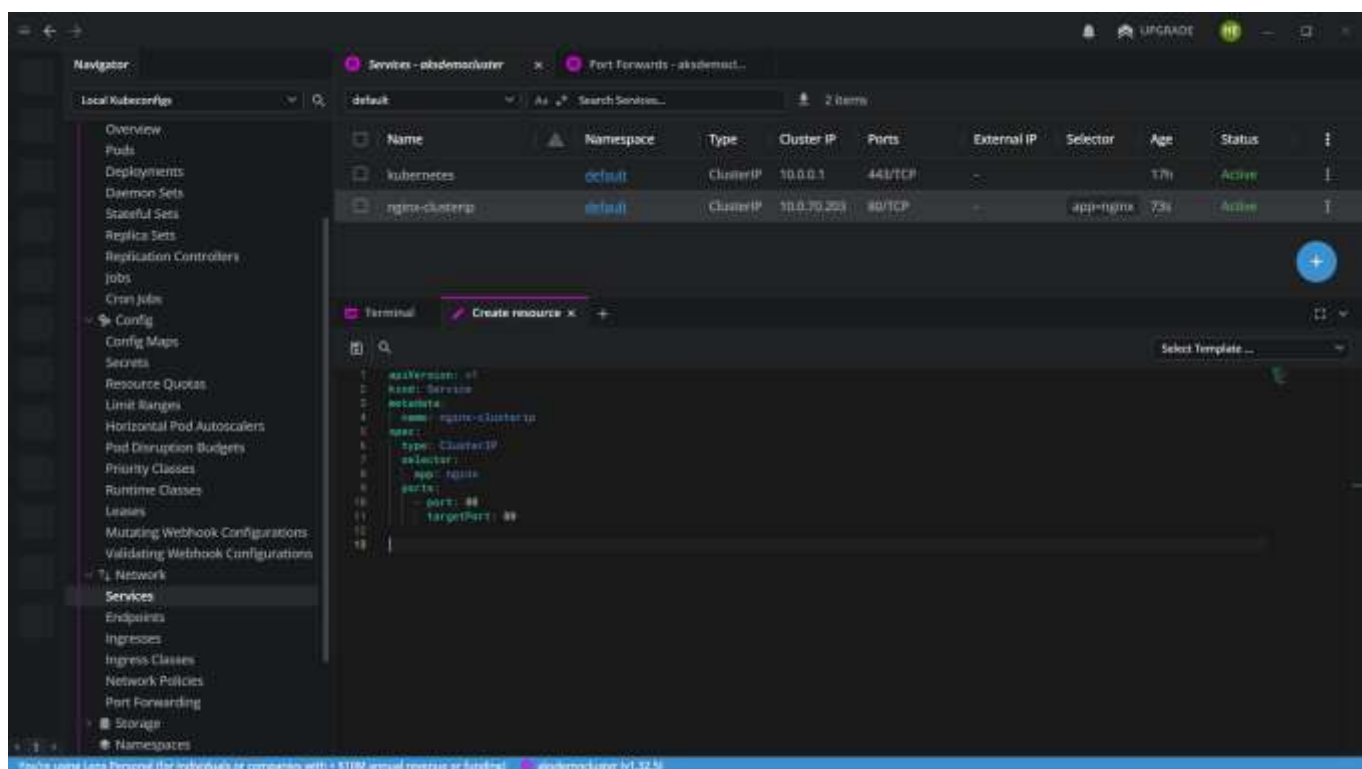
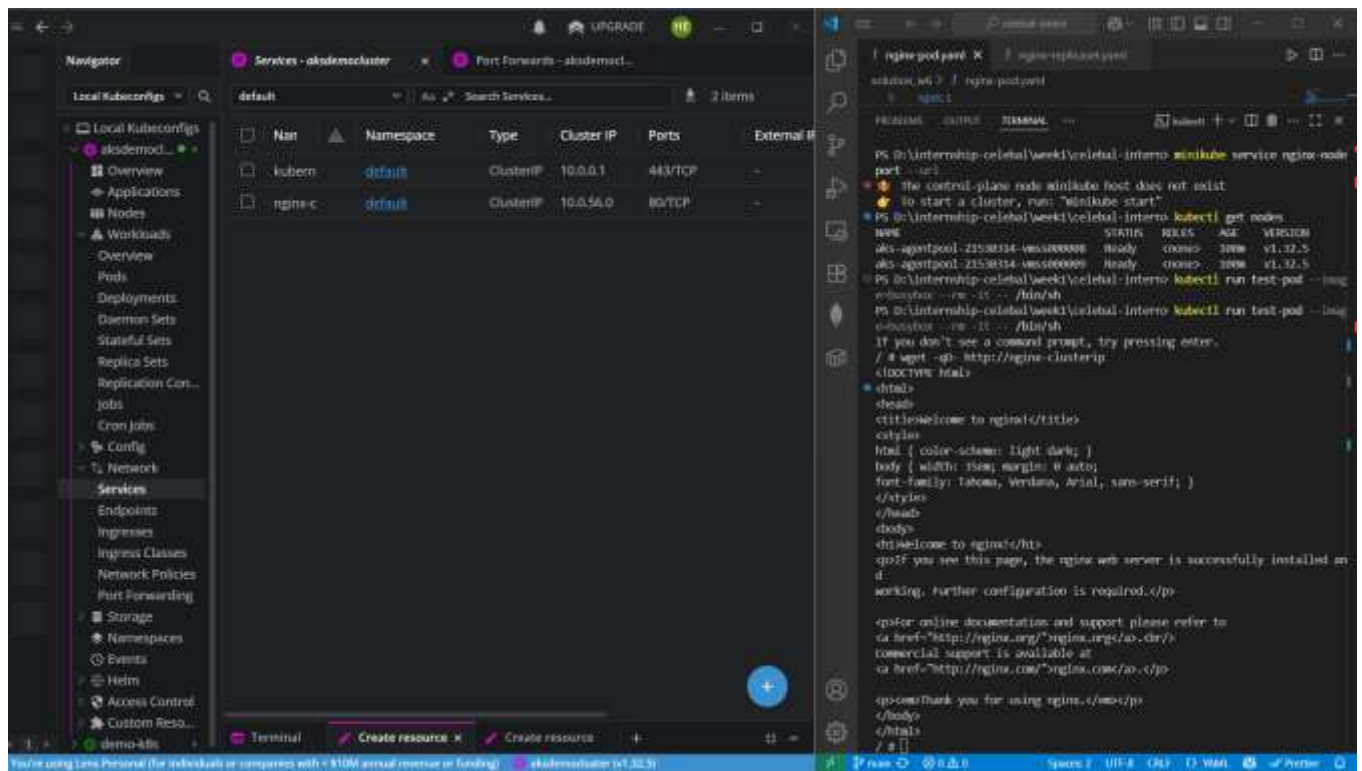
In Kubernetes, Services provide a stable network identity and DNS name for a set of pods. There are three main types of services for exposing applications:

1. ClusterIP (default):

- Exposes the service internally (within the cluster only).
- Pods can communicate with it, but not accessible from outside the cluster.
- Used for Internal microservices, backend-to-backend communication.

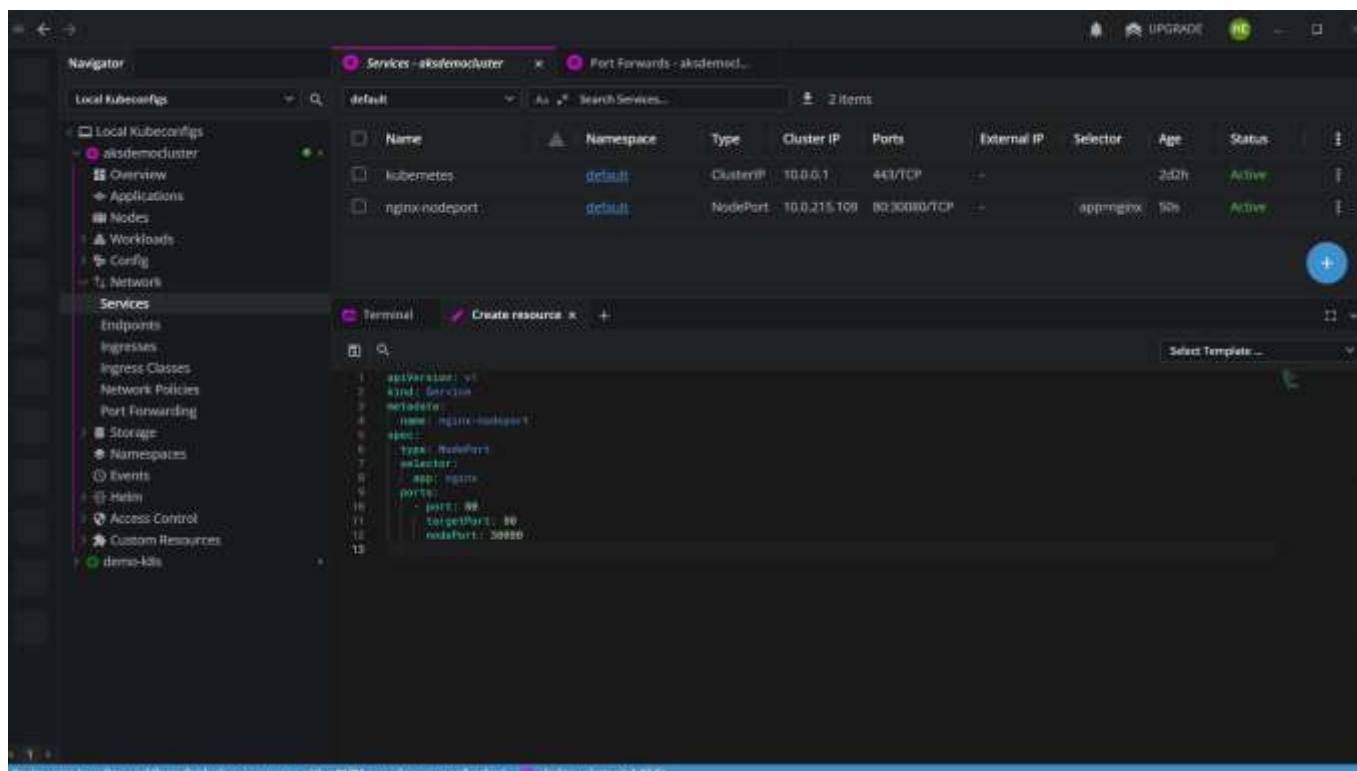
The screenshot shows a Kubernetes dashboard on the left and a terminal window on the right. The dashboard displays the 'Services' section for the 'aksdemo1' cluster. The terminal window shows the following commands and output:

```
PS C:\kubernetes\demo1> kubectl create -f nginx-service.yaml
service/nginx created
PS C:\kubernetes\demo1> kubectl get services
NAME      TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)
nginx     ClusterIP   10.10.10.1    <none>         80/TCP
PS C:\kubernetes\demo1> kubectl run test-pod --image nginx --port 80
pod/test-pod created
PS C:\kubernetes\demo1> kubectl exec test-pod -- curl -i http://nginx
HTTP/1.1 200 OK (text/html)
Date: Wed, 07 Nov 2018 14:14:14 GMT
Content-Type: text/html
Content-Length: 163
Server: Apache/2.4.18 (Ubuntu)
<html>
<head>
<title>Welcome to nginx!</title>
<style>
body { width: 100%; height: 100%; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<div style="display: flex; justify-content: space-between; align-items: center;">
<div>
Welcome to nginx!
</div>
<div>
If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.
</div>
</div>
<div style="display: flex; justify-content: space-between; align-items: center;">
<div>
<small>For more information, see the nginx documentation at<br>
http://nginx.org
</small>
</div>
<div>
<small>Commercial support is available at<br>
http://nginx.com
</small>
</div>
</div>
</body>
</html>
```

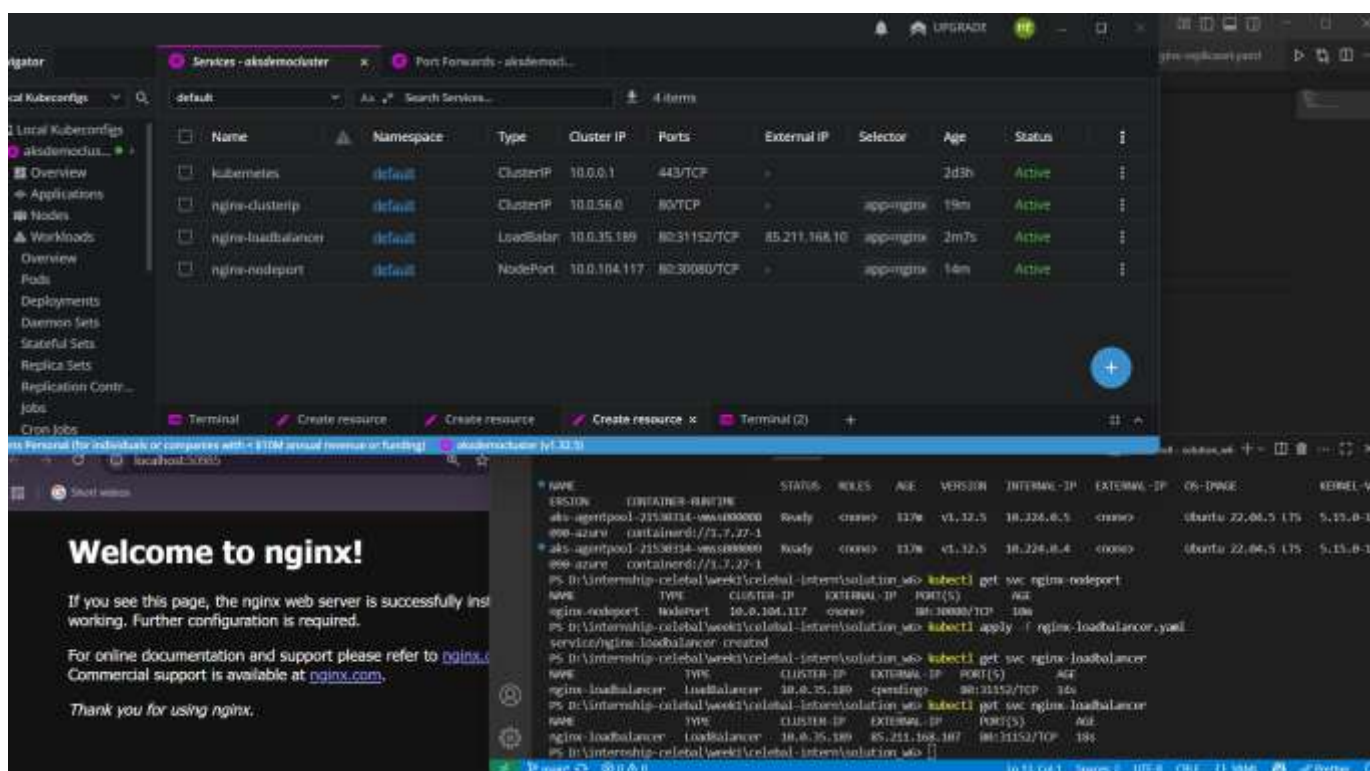
2. NodePort:

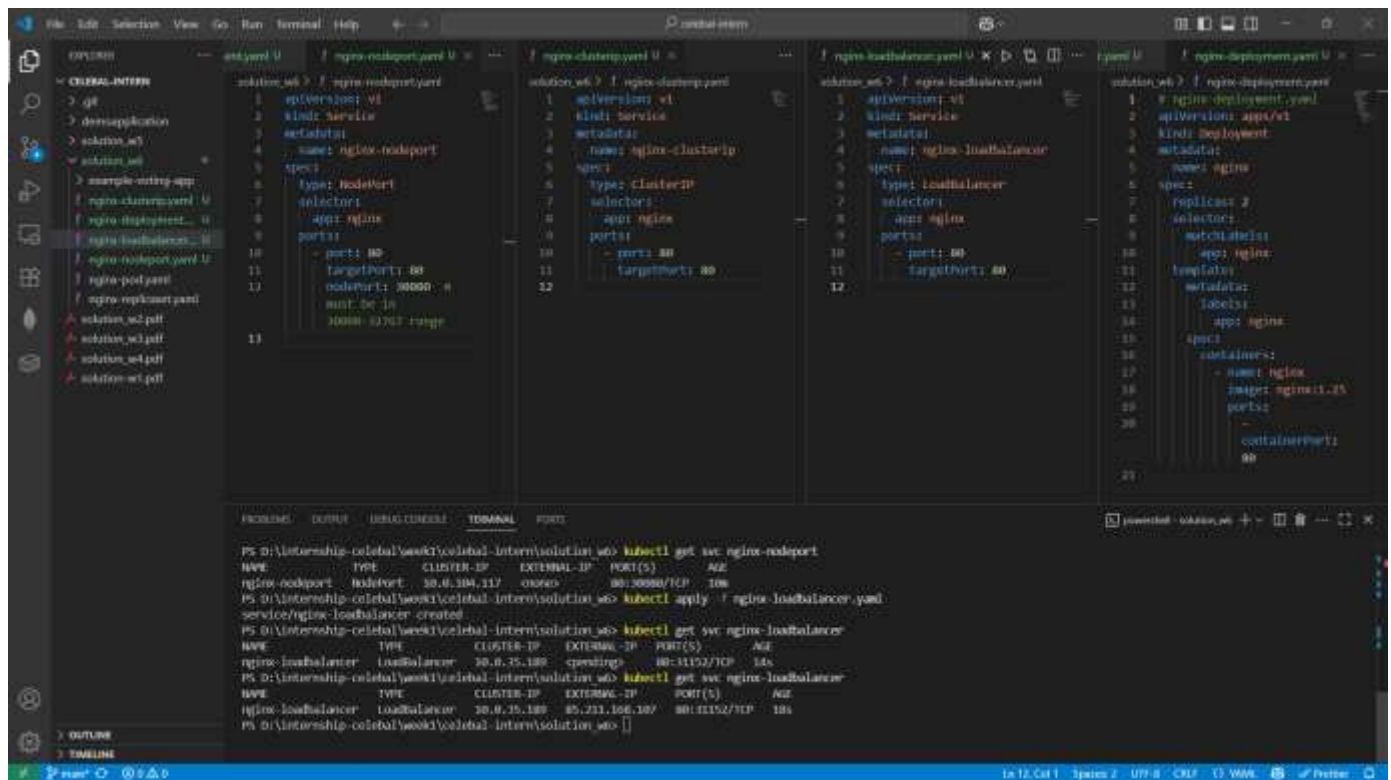
- Exposes the service on each Node's IP at a static port (30000–32767).
- Makes the service accessible from outside the cluster using.
- `http://<NodeIP>:<NodePort>`
- For development/testing environments where you want to quickly access an app from outside.



3. LoadBalancer:

- Provisions an external load balancer (supported in cloud platforms like Azure, AWS, GCP).
- Exposes service with a **public IP**.
- Automatically routes traffic to backend pods.
- Production-grade apps that need public access.





Q3. PersistentVolume (PV) and PersistentVolumeClaim (PVC)

What is a PersistentVolume (PV)?

A PersistentVolume is a piece of storage provisioned by an admin or dynamically provisioned using a StorageClass. It's like a USB drive available to the cluster.

What is a PersistentVolumeClaim (PVC)?

A PersistentVolumeClaim is a request for storage by a user (pod). Think of it like: "I want 1Gi of storage that supports ReadWriteOnce".

VS Code interface showing the Kubernetes dashboard and terminal output.

Left Panel (Explorer): Shows the file structure of the Kubernetes manifests, including `deployment.yaml`, `service.yaml`, `pod.yaml`, and `pv.yaml`.

Right Panel (Terminal): Displays the output of the `kubectl` commands. The output shows the creation of the `nginx-pvc` pod and the `nginx-pv` persistent volume claim.

Terminal Output:

```
PS D:\Internship-CELEBA\week1\celeba-intern\solution_w6> kubectl apply -f pv.yaml
persistentvolume/my-pv unchanged
PS D:\Internship-CELEBA\week1\celeba-intern\solution_w6> kubectl apply -f pvc.yaml
persistentvolumeclaim/my-pvc created
PS D:\Internship-CELEBA\week1\celeba-intern\solution_w6> kubectl apply -f pod-pvc.yaml
pod/nginx-pvc-pod created
PS D:\Internship-CELEBA\week1\celeba-intern\solution_w6> kubectl get pv
NAME          CAPACITY  ACCESS MODES  RECLAIM POLICY  STATUS    CLAIM              STORAGECLASS  VOLUMEATTRIBUTESCLASS  REASON   AGE
my-pv         1Gi       RWO           Retain          Available  default/my-pvc     default         <unset>    96s
pvc-2e32dc08-3767-4fad-8622-bfc2be7b0a0c  1Gi       RWO           Delete          Bound     default/my-pvc     default         <unset>    47s
PS D:\Internship-CELEBA\week1\celeba-intern\solution_w6> kubectl get pvc
NAME          STATUS    VOLUME          CAPACITY  ACCESS MODES  STORAGECLASS  VOLUMEATTRIBUTESCLASS  AGE
my-pvc        Bound     pvc-2e32dc08-3767-4fad-8622-bfc2be7b0a0c  1Gi       RWO           default       <unset>              260s
```

VS Code interface showing the Kubernetes dashboard and terminal output.

Left Panel (Explorer): Shows the file structure of the Kubernetes manifests, including `deployment.yaml`, `service.yaml`, `pod.yaml`, and `pv.yaml`.

Right Panel (Terminal): Displays the output of the `kubectl` commands. The output shows the creation of the `nginx-pvc` pod and the `nginx-pv` persistent volume claim.

Terminal Output:

```
PS D:\Internship-CELEBA\week1\celeba-intern\solution_w6> kubectl apply -f pv.yaml
persistentvolume/my-pv unchanged
PS D:\Internship-CELEBA\week1\celeba-intern\solution_w6> kubectl apply -f pvc.yaml
persistentvolumeclaim/my-pvc created
PS D:\Internship-CELEBA\week1\celeba-intern\solution_w6> kubectl apply -f pod-pvc.yaml
pod/nginx-pvc-pod created
PS D:\Internship-CELEBA\week1\celeba-intern\solution_w6> kubectl get pv
NAME          CAPACITY  ACCESS MODES  RECLAIM POLICY  STATUS    CLAIM              STORAGECLASS  VOLUMEATTRIBUTESCLASS  REASON   AGE
my-pv         1Gi       RWO           Retain          Available  default/my-pvc     default         <unset>    96s
pvc-2e32dc08-3767-4fad-8622-bfc2be7b0a0c  1Gi       RWO           Delete          Bound     default/my-pvc     default         <unset>    47s
PS D:\Internship-CELEBA\week1\celeba-intern\solution_w6> kubectl get pvc
NAME          STATUS    VOLUME          CAPACITY  ACCESS MODES  STORAGECLASS  VOLUMEATTRIBUTESCLASS  AGE
my-pvc        Bound     pvc-2e32dc08-3767-4fad-8622-bfc2be7b0a0c  1Gi       RWO           default       <unset>              260s
```

VS Code interface showing the Kubernetes dashboard and terminal output.

Left Panel (Explorer): Shows the file structure of the Kubernetes manifests, including `deployment.yaml`, `service.yaml`, `pod.yaml`, and `pv.yaml`.

Right Panel (Terminal): Displays the output of the `kubectl` commands. The output shows the creation of the `nginx-pvc` pod and the `nginx-pv` persistent volume claim.

Terminal Output:

```
PS D:\Internship-CELEBA\week1\celeba-intern\solution_w6> kubectl apply -f pv.yaml
persistentvolume/my-pv unchanged
PS D:\Internship-CELEBA\week1\celeba-intern\solution_w6> kubectl apply -f pvc.yaml
persistentvolumeclaim/my-pvc created
PS D:\Internship-CELEBA\week1\celeba-intern\solution_w6> kubectl apply -f pod-pvc.yaml
pod/nginx-pvc-pod created
PS D:\Internship-CELEBA\week1\celeba-intern\solution_w6> kubectl get pv
NAME          CAPACITY  ACCESS MODES  RECLAIM POLICY  STATUS    CLAIM              STORAGECLASS  VOLUMEATTRIBUTESCLASS  REASON   AGE
my-pv         1Gi       RWO           Retain          Available  default/my-pvc     default         <unset>    96s
pvc-2e32dc08-3767-4fad-8622-bfc2be7b0a0c  1Gi       RWO           Delete          Bound     default/my-pvc     default         <unset>    47s
PS D:\Internship-CELEBA\week1\celeba-intern\solution_w6> kubectl get pvc
NAME          STATUS    VOLUME          CAPACITY  ACCESS MODES  STORAGECLASS  VOLUMEATTRIBUTESCLASS  AGE
my-pvc        Bound     pvc-2e32dc08-3767-4fad-8622-bfc2be7b0a0c  1Gi       RWO           default       <unset>              260s
```

Q4. Managing Kubernetes with Azure Kubernetes Service (AKS), Creating and managing AKS clusters, Scaling and upgrading AKS clusters

AKS (Azure Kubernetes Service) is Microsoft's fully managed Kubernetes service.

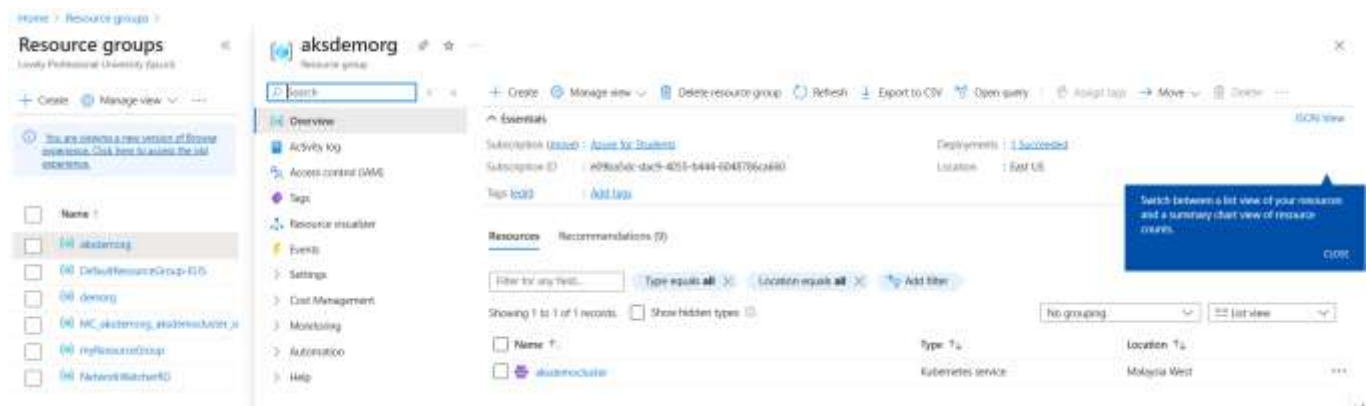
Think of it like this:

"You want to use Kubernetes but don't want to handle all the hard setup, updates, and server maintenance — Azure does it for you."

Step1: Creating an AKS Cluster or we can use azure command line also

Using Azure Portal (GUI)

1. Go to <https://portal.azure.com>
2. Search for "Kubernetes services" → Click **Create**
3. Fill in details:
 - **Resource Group:** Like a folder for resources.
 - **Cluster Name:** e.g., my-aks-cluster
 - **Region:** e.g., Central India
 - **Node Size & Count:** Like choosing how powerful and how many VMs you want.
4. Hit **Review + Create**, and Azure will set it all up for you in a few minutes.



Step2: Managing the Cluster



Step3: Scaling the Cluster means we can set it as autoscaling or scaling. In autoscaling we can scale it by giving it minimum and maximum nodes so that when ever new node is required by the aks it can create without any interference. But in scaling we manually add new node to manage the load.

```
PS D:\Internship-celebal\week1\celebal-intern\solution_w6> az aks update --resource-group aksdemorg --name aksdemocluster --enable-cluster-autoscaler --min-count 2 --max-count 5
{
  "aadProfile": null,
  "addonProfiles": {
    "azureKeyVaultSecretsProvider": {
      "config": null,
      "enabled": false,
      "identity": null
    },
    "azurepolicy": {
      "config": null,
      "enabled": false,
      "identity": null
    }
  }
}
```

Step4: Upgrading to latest version

note: Azure does a **rolling upgrade**, so your workloads keep running without downtime.

```
PS D:\Internship-celebal\week1\celebal-intern\solution_w6> az aks get-upgrades --resource-group aksdemorg --name aksdemocluster
{
  "agentPoolProfiles": null,
  "controlPlaneProfile": {
    "kubernetesVersion": "1.32.5",
    "name": null,
    "osType": "linux",
    "upgrades": [
      {
        "isPreview": null,
        "kubernetesVersion": "1.33.1"
      },
      {
        "isPreview": null,
        "kubernetesVersion": "1.33.0"
      }
    ]
  },
  "id": "/subscriptions/e09ba5dc-dac9-4055-b444-6048706ca668/resourcegroups/aksdemorg/providers/Microsoft.ContainerService/managedClusters/aksdemocluster/upgradeProfiles/default",
  "name": "default",
  "resourceGroup": "aksdemorg",
  "type": "Microsoft.ContainerService/managedClusters/upgradeProfiles"
}
```

```
PS D:\Internship-celebal\week1\celebal-intern\solution_w6> az aks upgrade --resource-group aksdemorg --name aksdemocluster --kubernetes-version 1.32.5
Kubernetes may be unavailable during cluster upgrades.
Are you sure you want to perform this operation? (y/N): y
The cluster is already on version 1.32.5 and is not in a failed state. No operations will occur when upgrading to the same version if the cluster is not in a failed state.
Since control-plane-only argument is not specified, this will upgrade the control plane AND all nodepools to version 1.32.5. Continue? (y/N): n
PS D:\Internship-celebal\week1\celebal-intern\solution_w6>
```

Q5. Configure liveness and readiness probes for pods in AKS cluster.

Before we start, let's understand the **why**.

- **Liveness Probe** checks if the app is still running. If it's not, Kubernetes restarts the container.
- **Readiness Probe** checks if the app is ready to serve requests. If it's not ready, Kubernetes removes the pod from service load balancers until it is.

This helps keep apps highly available, stable, and self-healing.

Step 1: Connect to the AKS Cluster

First, I connect my terminal to the AKS cluster using the following command:

```
az aks get-credentials --resource-group myResourceGroup --name myAKSCluster
```

This command downloads the credentials and sets my kubectl context to AKS.

Step 2: Create a Deployment with Probes

Now, I create a new deployment named nginx-probe-demo. But this time, I'm going to add **both liveness and readiness probes** in the YAML definition.

This means the **livenessProbe** starts checking 10 seconds after the container starts and repeats every 5 seconds. If it fails, the container is restarted and the **readinessProbe** checks after 5 seconds to see if the container is ready to serve traffic.

Step 3: Apply the Deployment

Step 4: Monitor the Probes in Action : If the **readiness probe failed**, the pod won't receive traffic and if the **liveness probe failed**, the pod will be restarted. In the output, I check the events section. It will show me:

```
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q5> cd ../Q5
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q5> kubectl apply -f nginx-probe.yaml
deployment.apps/nginx-probe-demo created
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q5> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
nginx-probe-demo-745cd6b74f-95kbz   1/1     Running   0           113s
nginx-probe-demo-745cd6b74f-kx8kl   1/1     Running   0           113s
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q5> kubectl describe pod nginx-probe-demo-745cd6b74f-95kbz
Name:                               nginx-probe-demo-745cd6b74f-95kbz
Namespace:                           default
Priority:                              0
Service Account:                       default
Node:                                 aks-agentpool-21530314-vms00000c/10.244.0.4
Start Time:                           Wed, 16 Jul 2025 10:40:34 +0530
Labels:                                app=nginx-probe
                                         pod-template-hash=745cd6b74f
Annotations:                           <none>
Status:                                Running
IP:                                    10.244.1.23
IPs:                                   IP: 10.244.1.23
Controlled By:                         ReplicaSet/nginx-probe-demo-745cd6b74f
Containers:
  nginx:
    container ID:  containerd://3fbb10c28311a4555c19835e5ad6b6d801ff7033f76097dd872f60d7e7d8bba657
```

```
Events:
  Type     Reason      Age    From          Message
  ----     -
  Normal   Scheduled   2m69s  default-scheduler  Successfully assigned default/nginx-probe-demo-745cd6b74f-95kbz to aks-agentpool-21530314-vms00000c
  Normal   Pulling     2m69s  kubelet         Pulling image "nginx"
  Normal   Pulled      2m68s  kubelet         Successfully pulled image "nginx" in 8.719s (8.719s including waiting). Image size: 72223778 bytes.
  Normal   Created     2m68s  kubelet         Created container: nginx
  Normal   Started     2m68s  kubelet         Started container nginx
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q5>
```

Step 5 : Simulate Failure - To test the probe in action, I could run the below command and then observe the logs and pod restarts

```
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q5> kubectl exec -i nginx-probe-demo-745cd6b74f-95kbz -- /bin/sh
# rm -rf /usr/share/nginx/html/index.html
# exit
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q5> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
nginx-probe-demo-745cd6b74f-95kbz   0/1     Running   0           10m
nginx-probe-demo-745cd6b74f-kx8kl   1/1     Running   0           10m
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q5> kubectl describe pod nginx-probe-demo-745cd6b74f-95kbz
Name:                               nginx-probe-demo-745cd6b74f-95kbz
Namespace:                           default
Priority:                              0
Service Account:                       default
Node:                                 aks-agentpool-21530314-vms00000c/10.244.0.4
Start Time:                           Wed, 16 Jul 2025 10:40:34 +0530
Labels:                                app=nginx-probe
                                         pod-template-hash=745cd6b74f
Annotations:                           <none>
Status:                                Running
IP:                                    10.244.1.23
IPs:                                   IP: 10.244.1.23
Controlled By:                         ReplicaSet/nginx-probe-demo-745cd6b74f
Containers:
  nginx:
    container ID:  containerd://3fbb10c28311a4555c19835e5ad6b6d801ff7033f76097dd872f60d7e7d8bba657
    image:          nginx:1.25.3
    image pull spec: nginx:1.25.3
    ports:
      - containerPort: 80
    liveness:
      enabled: true
      httpGet:
        path: /
        port: 80
      initialDelaySeconds: 10
      periodSeconds: 5
    readiness:
      enabled: true
      httpGet:
        path: /
        port: 80
      initialDelaySeconds: 5
      periodSeconds: 5
    resources:
      requests:
        cpu: 100m
        memory: 128Mi
    terminationMessagePath: /dev/termination-log
    volumeMounts:
      - mountPath: /usr/share/nginx/html
        name: nginx-data
    restartPolicy: Always
    startTimestamp: 2025-07-16T05:40:34Z
    status: Running
    system ID: 3fbb10c28311a4555c19835e5ad6b6d801ff7033f76097dd872f60d7e7d8bba657
    volume: nginx-data
Events:
  Type     Reason      Age    From          Message
  ----     -
  Normal   Scheduled   10m    default-scheduler  Successfully assigned default/nginx-probe-demo-745cd6b74f-95kbz to aks-agentpool-21530314-vms00000c
  Normal   Pulling     10m    kubelet         Pulling image "nginx"
  Warning  Unhealthy   27s (x2 over 32s)  kubelet         Readiness probe failed: HTTP probe failed with statuscode: 403
  Normal   Pulling     23s (x2 over 10m)  kubelet         Pulling image "nginx"
  Warning  Unhealthy   23s (x3 over 33s)  kubelet         Liveness probe failed: HTTP probe failed with statuscode: 403
  Normal   Killing     23s      kubelet         Container nginx failed liveness probe, will be restarted
  Warning  Unhealthy   22s      kubelet         Readiness probe failed: Get "http://10.244.1.23:80/": dial tcp 10.244.1.23:80: connect: connection refused
  Normal   Created     21s (x2 over 10m)  kubelet         Created container: nginx
  Normal   Started     21s (x2 over 10m)  kubelet         Started container nginx
  Normal   Pulled      21s      kubelet         Successfully pulled image "nginx" in 1.934s (1.934s including waiting). Image size: 72223778 bytes.
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q5>
```

- With liveness probes, Kubernetes ensures that crashed containers get restarted.

- With readiness probes, Kubernetes ensures that traffic goes only to healthy pods.
- These probes are critical in production environments to maintain high uptime and graceful scaling.

Q6. Configure Taints and Tolerations.

What Are Taints and Tolerations?

Let me explain this simply:

- A **taint** is something we apply to a **node** to say:

"Don't schedule any pods here unless they can tolerate this condition."

- A **toleration** is something we apply to a **pod** to say:

"It's okay, I can run on a node with this taint."

This helps us **control which pods can run on which nodes**, useful for:

- Running sensitive workloads separately
- Isolating GPU/ML nodes
- Separating environments (e.g., dev vs prod)

Step 1: Get Your Node Names - First, I list the nodes in my AKS cluster: "kubectl get nodes". I pick one of the node names,

Step 2: Apply a Taint to the Node

```
PS D:\Internship-celebal\week1\celebal-intern\solution_w6\Q5> kubectl get nodes
NAME                                STATUS    ROLES    AGE   VERSION
aks-agentpool-21530314-vmss00000c  Ready    <none>   38m   v1.32.5
aks-agentpool-21530314-vmss00000d  Ready    <none>   38m   v1.32.5
PS D:\Internship-celebal\week1\celebal-intern\solution_w6\Q5> kubectl taint nodes aks-agentpool-21530314-vmss00000c key=exclusive:NoSchedule
error: invalid taint effect: NoSchedule, unsupported taint effect
See 'kubectl taint -h' for help and examples
PS D:\Internship-celebal\week1\celebal-intern\solution_w6\Q5> kubectl taint nodes aks-agentpool-21530314-vmss00000c key=exclusive:NoSchedule
node/aks-agentpool-21530314-vmss00000c tainted
PS D:\Internship-celebal\week1\celebal-intern\solution_w6\Q5> █
```

This tells Kubernetes **not to schedule pods on this node** unless they **tolerate** key=exclusive

Step 3: Try Running a Pod Without Toleration

```
PS D:\Internship-celebal\week1\celebal-intern\solution_w6\Q6> kubectl apply -f pod-no-toleration.yaml
pod/nginx-no-toleration created
PS D:\Internship-celebal\week1\celebal-intern\solution_w6\Q6> kubectl get pods
NAME                                READY    STATUS    RESTARTS   AGE
nginx-no-toleration                 1/1      Running   0           13s
nginx-probe-demo-745cd6b74f-95kbr  1/1      Running   0           18m
nginx-probe-demo-745cd6b74f-kc8kl  1/1      Running   0           18m
PS D:\Internship-celebal\week1\celebal-intern\solution_w6\Q6> kubectl describe pod nginx-no-toleration
Name:                               nginx-no-toleration
Namespace:                           default
Priority:                             0
Service Account:                      default
Node:                                 aks-agentpool-21530314-vmss00000d/10.244.0.5
Start Time:                           Wed, 16 Jul 2025 10:59:07 +0530
Labels:                               <none>
Annotations:                           <none>
Status:                               Running
IP:                                   10.244.0.104
IPs:
  IP: 10.244.0.104
Containers:
  nginx:
    container ID:  containerd://ab0804c504750d28b9fd397d5673fddeb44243d4a485130f474ec65e554d4e1e
    Image:           nginx
    Image ID:        docker.io/library/nginx@sha256:f5c017fb33c6db484545793ffb67db51cdd7daebee472104612f73a85063f889
```



```

Node-Selectors:      <none>
Tolerations:         node.kubernetes.io/not-ready:NoExecute op-Exists for 300s
                     node.kubernetes.io/unreachable:NoExecute op-Exists for 300s
Events:
  Type     Reason      Age   From          Message
  ----     -
  Normal   Scheduled   13s   default-scheduler   Successfully assigned default/nginx-no-toleration to aks-agentpool-21530314-vmss00000d
  Normal   Pulling     13s   kubelet         Pulling image "nginx"
  Normal   Pulled      11s   kubelet         Successfully pulled image "nginx" in 1.914s (1.914s including waiting). Image size: 72223778 bytes.
  Normal   Created     11s   kubelet         Created container: nginx
  Normal   Started     11s   kubelet         Started container nginx
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q6>

```

Output of kubectl describe pod showing the "**node(s) had taints**" error.

Step 4: Add Toleration to Allow Scheduling

Output of kubectl get pods -o wide showing the pod running

```

PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q6> kubectl apply -f pod-with-toleration.yaml
pod/nginx-with-toleration created
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q6> kubectl get pods -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE                                NOMINATED NODE   READINESS GATES
nginx-no-toleration                 1/1     Running   0           4m28s  10.244.0.184    aks-agentpool-21530314-vmss00000d  <none>            <none>
nginx-probe-demo-745cd6b74f-95kdy  1/1     Running   1 (12m ago)  23m   10.244.1.23     aks-agentpool-21530314-vmss00000d  <none>            <none>
nginx-probe-demo-745cd6b74f-kozkl  1/1     Running   0           23m   10.244.0.285    aks-agentpool-21530314-vmss00000d  <none>            <none>
nginx-with-toleration               1/1     Running   0           20s    10.244.0.232    aks-agentpool-21530314-vmss00000d  <none>            <none>
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q6> kubectl describe pod nginx-with-toleration
Name:          nginx-with-toleration
Namespace:     default
Priority:       0
Service Account: default
Node:          aks-agentpool-21530314-vmss00000d/10.244.0.5
Start Time:    Wed, 16 Jul 2025 11:03:15 +0530
Labels:        <none>
Annotations:   <none>
Status:        Running
IP:            10.244.0.232
IPs:           IP: 10.244.0.232
Containers:
  nginx:
    Container ID:  containerd://e680ec402f077c815089a421d5e5e972d03117d8c681b3ebc8f8d0055fc
    Image:         nginx
    Image ID:      docker.io/library/nginx@sha256:f5c817fb33c6db484545793ffb67db51cdd7daebee472104612f73a85063f889
    Port:         <none>
    Host Port:     <none>
    State:         Running
Node-Selectors:      <none>
Tolerations:         exclusive=true:NoSchedule
                     node.kubernetes.io/not-ready:NoExecute op-Exists for 300s
                     node.kubernetes.io/unreachable:NoExecute op-Exists for 300s
Events:
  Type     Reason      Age   From          Message
  ----     -
  Normal   Scheduled   35s   default-scheduler   Successfully assigned default/nginx-with-toleration to aks-agentpool-21530314-vmss00000d
  Normal   Pulling     35s   kubelet         Pulling image "nginx"
  Normal   Pulled      33s   kubelet         Successfully pulled image "nginx" in 1.912s (1.912s including waiting). Image size: 72223778 bytes.
  Normal   Created     33s   kubelet         Created container: nginx
  Normal   Started     33s   kubelet         Started container nginx
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q6>

```

Output of kubectl describe pod nginx-with-toleration

Step 5: Remove Taint (Clean Up)

```

PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q6> kubectl taint nodes aks-agentpool-21530314-vmss00000d key-exclusive:NoSchedule-
node/aks-agentpool-21530314-vmss00000d untainted
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q6>

```

- I **tainted** a node so that pods won't go there unless they tolerate it.
- I showed how a pod without toleration fails to schedule.
- I added a **toleration** to a new pod and showed that it runs fine on the tainted node.
- This is useful when we want **strict control over pod placement**, like for **security**, **performance**, or **isolation**.

Q7. Create and attach persistent volume claims to pods.

Imagine you're running an app that stores data — like logs, user uploads, or configs. If the pod crashes or restarts, that data will be lost unless you attach persistent storage.

A PersistentVolumeClaim (PVC) lets your pod request and use persistent storage.

Step 1: Create a Persistent Volume (PV) - This step defines **where** the storage physically exists.

```
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q6> cd ../Q7
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q7> new-item pv.yaml

Directory: D:\internship-celebal\week1\celebal-intern\solution_w6\Q7

Mode                LastWriteTime         Length Name
----                -
-a-----          16-07-2025   11:09             0 pv.yaml

PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q7> kubectl apply -f pv.yaml
persistentvolume/my-pv created
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q7> kubectl get pv
NAME      CAPACITY   ACCESS MODES   RECLAIM POLICY   STATUS   CLAIM   STORAGECLASS   VOLUMEATTRIBUTESCLASS   REASON   AGE
my-pv     1Gi        RWO            Retain           Available             Available              <unset>              75s
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q7> |
```

Step 2: Create a Persistent Volume Claim (PVC)

```
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q7> kubectl delete pvc my-pvc
persistentvolumeclaim "my-pvc" deleted
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q7> kubectl delete pv my-pv
persistentvolume "my-pv" deleted
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q7>
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q7> kubectl apply -f pv.yaml
persistentvolume/my-pv created
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q7> kubectl apply -f pvc.yaml
persistentvolumeclaim/my-pvc created
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q7> kubectl get pvc
NAME      STATUS   VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
my-pvc    Bound    my-pv    1Gi        RWO            <unset>         <unset>                 11s
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q7> kubectl get pv
NAME      CAPACITY   ACCESS MODES   RECLAIM POLICY   STATUS   CLAIM   STORAGECLASS   VOLUMEATTRIBUTESCLASS   REASON   AGE
my-pv     1Gi        RWO            Retain           Bound    default/my-pvc    default/my-pvc    <unset>                 22s
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q7> |
```

Step 3: Attach PVC to a Pod

Let's check if the volume is mounted and working.

```
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q7> kubectl apply -f pod-pvc.yaml
pod/pvc-nginx created
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q7> kubectl get pods
NAME      READY   STATUS    RESTARTS   AGE
pvc-nginx 1/1     Running   0          12s
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q7> kubectl describe pod pvc-nginx
Name:      pvc-nginx
Namespace: default
Priority:   0
Service Account: default
Node:      aks-agentpool-21530314-vmss00000d/10.244.0.5
Start Time: Wed, 16 Jul 2025 12:32:48 +0530
Labels:    <none>
Annotations: <none>
Status:    Running
IP:        10.244.0.62
IPs:
  IP: 10.244.0.62
Containers:
  nginx:
    Container ID:   containerd://f01b772f648a4a594956bf0b58dc7ac7a4519a2cd173b36d83a2f9d3cf222013
    Image:          nginx
    Image ID:       docker.io/library/nginx@sha256:f5c017fb33c6db484545793ffb67db51cdd7daebef472104612f73a85063f889
    Port:           <none>
```

```

QoS Class:           BestEffort
Node-Selectors:      <none>
Tolerations:         node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                     node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type     Reason      Age   From          Message
  ----     ------      -
  Normal   Scheduled   20s   default-scheduler   Successfully assigned default/pvc-nginx to aks-agentpool-21530314-vmss00000d
  Normal   Pulling     20s   kubelet          Pulling image "nginx"
  Normal   Pulled      18s   kubelet          Successfully pulled image "nginx" in 1.837s (1.837s including waiting). Image size: 72223776 bytes.
  Normal   Created     18s   kubelet          Created container: nginx
  Normal   Started     18s   kubelet          Started container: nginx
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q7> kubectl exec -it pvc-nginx -- /bin/bash
root@pvc-nginx:/# cat index.html
cat: index.html: No such file or directory
root@pvc-nginx:/# cd /usr/share/nginx/html
root@pvc-nginx:/usr/share/nginx/html# echo "Hello from PVC" > index.html
root@pvc-nginx:/usr/share/nginx/html# cat index.html
Hello from PVC

```

Terminal showing file creation and content from mounted volume

Q8. Configure health probes for pods.

When you're running applications in Kubernetes, you need a way to **check if your application is running properly**.

Kubernetes uses two types of health checks (probes):

- **Liveness Probe:**
Checks **if the app is still alive**. If it fails, Kubernetes **restarts** the container.
- **Readiness Probe:**
Checks **if the app is ready to serve traffic**. If it fails, the pod is **removed from the load balancer** temporarily.

This keeps your app **self-healing** and **resilient**.

Step 1: Create a Pod with Liveness and Readiness Probe

I'm creating an NGINX pod that exposes a simple HTTP service, and I'm adding both types of health probes to it.

```

• PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q7> cd ../Q8
• PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q8> kubectl apply -f nginx-probes.yaml
pod/nginx-health-demo created
• PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q8> kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
nginx-health-demo    1/1     Running   0           26s
pvc-nginx           1/1     Running   0           6m2s
• PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q8> kubectl describe pod nginx-health-demo
Name:                nginx-health-demo
Namespace:            default
Priority:              0
Service Account:      default
Node:                 aks-agentpool-21530314-vmss00000d/10.224.0.5
Start Time:           Wed, 16 Jul 2025 12:38:24 +0530
Labels:               <none>
Annotations:          <none>
Status:               Running
IP:                   10.244.0.12
IPs:

```

```

ConfigMapName: kube-root-ca.crt
Optional:      false
DownwardAPI:  true
QoS Class:     BestEffort
Node-Selectors: <none>
Tolerations:   node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
               node.kubernetes.io/unreachable:NoExecute op=Exists for 300s

Events:
  Type    Reason      Age   From          Message
  ----    -
  Normal  Scheduled   33s   default-scheduler Successfully assigned default/nginx-health-demo to aks-agentpool-21530314-vmss00000d
  Normal  Pulling     32s   kubelet       Pulling image "nginx"
  Normal  Pulled      30s   kubelet       Successfully pulled image "nginx" in 1.92s (1.92s including waiting). Image size: 72223778 bytes.
  Normal  Created     30s   kubelet       Created container: nginx
  Normal  Started     30s   kubelet       Started container nginx

```

Liveness Probe starts after 10s and runs every 5s. If it fails, the container restarts.

Readiness Probe starts after 5s and runs every 5s. If it fails, the pod is marked "Not Ready" and is temporarily removed from load balancing.

Q9. Configure autoscaling in your cluster (Horizontal scaling)

Horizontal Pod Autoscaler (HPA) automatically adjusts the number of pod replicas based on **CPU usage** or other custom metrics.

Think of it like this:

- When traffic is low, run fewer pods to save resources.
- When traffic spikes, automatically increase the number of pods to handle it.

HPA helps your app scale **dynamically**, without manual intervention.

Step 1: Enable Metrics Server (Required)

```

PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q9> kubectl get deployment metrics-server -n kube-system
NAME          READY  UP-TO-DATE  AVAILABLE  AGE
metrics-server 2/2    2           2          5d2h
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q9>

```

Step 2: Create a Sample Deployment with CPU Limits

We'll use a simple NGINX deployment but add CPU **requests and limits** — required for autoscaling.

```

PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q9> kubectl apply -f nginx-deployment-hpa.yaml
deployment.apps/nginx-hpa created
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q9>

```

Step 3: Create the Horizontal Pod Autoscaler

Step 4: View HPA Status

```

PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q9> kubectl get hpa
NAME          REFERENCE          TARGETS          MINPODS  MAXPODS  REPLICAS  AGE
nginx-hpa     Deployment/nginx-hpa  cpu: <unknown>/50%  1         5         0         12s
PS D:\internship-celebal\week1\celebal-intern\solution_w6\Q9> kubectl run -i --tty load-generator --rm --image=busybox /bin/sh
If you don't see a command prompt, try pressing enter.
/ # while true; do wget -q -O- http://nginx-hpa; done

```

Step 5: Simulate High CPU Load

Step 6: Stop Load and Watch Scale Down

```
PS D:\internship-celebal\week1\celebal-intern> kubectl get hpa
NAME          REFERENCE          TARGETS      MINPODS  MAXPODS  REPLICAS  AGE
nginx-hpa     Deployment/nginx-hpa  cpu: 0%/50%    1         5         1         4m52s
PS D:\internship-celebal\week1\celebal-intern> kubectl get pods
NAME          READY   STATUS    RESTARTS  AGE
load-generator 1/1     Running   0          41s
nginx-health-demo 1/1     Running   0          9m58s
nginx-hpa-5b6dcf667b-tzx4g 1/1     Running   0          6m4s
pvc-nginx      1/1     Running   0          15m
PS D:\internship-celebal\week1\celebal-intern> kubectl get hpa
NAME          REFERENCE          TARGETS      MINPODS  MAXPODS  REPLICAS  AGE
nginx-hpa     Deployment/nginx-hpa  cpu: 0%/50%    1         5         1         4m53s
PS D:\internship-celebal\week1\celebal-intern> kubectl get pods
NAME          READY   STATUS    RESTARTS  AGE
load-generator 1/1     Running   0          43s
nginx-health-demo 1/1     Running   0          10m
nginx-hpa-5b6dcf667b-tzx4g 1/1     Running   0          6m6s
pvc-nginx      1/1     Running   0          15m
PS D:\internship-celebal\week1\celebal-intern> 
```

I configured **Horizontal Pod Autoscaling (HPA)** in my cluster using CPU metrics.

I added resource limits to the deployment, enabled metrics-server, and set autoscaling rules.

Then I stressed the pod to simulate high load and showed how it **scaled up automatically**.

Finally, when the load dropped, the pod count scaled back down, demonstrating autoscaling in action.